

CPCS 202 – Programming I

02 – Datatypes and Operators

Objectives

- Primitive Datatypes in Java
- String class in Java
- Operators

Output Statements in Java

- Java uses the following statement to display output to the console

```
System.out.println();
```

- Text or numbers to be displayed have to be inserted within the brackets
- Text to be displayed has to be specified within double quotes
 - For example `System.out.println("Welcome to Java");`
- The `+` symbol is used to concatenate (join) text
 - For example `System.out.println("Welcome to " + "Java");`

Output Statements in Java – More examples

```
System.out.println("Welcome to CPCS 202");  
System.out.println("Welcome to " + "CPCS 202");  
System.out.println("Area of circle with radius " + 12.5 + " is " + 490.87);  
System.out.println("5 + 3 = (5+3)");  
System.out.println("5 + 3 = " + (5+3));  
System.out.println("Price of 30 eggs (with VAT) is SAR " + 25.50);
```

Output

```
Welcome to CPCS 202  
Welcome to CPCS 202  
Area of circle with radius 12.5 is 490.87  
5 + 3 = (5+3)  
5 + 3 = 8  
Price of 30 eggs (with VAT) is SAR 25.5
```

Variables and Identifiers in Java

- Java uses **variables** to store numbers and text
- All Java variables must be identified with unique names known as identifiers
- Identifiers can be short names (like a, x and y) or more descriptive names (age, total, area, name)
- Identifiers in Java have to follow certain conventions (rules)

Rules for naming variables

- Names can only contain letters(a-z and A-Z), digits (0-9), underscores(_), and dollar signs (\$)
- Names can only begin with a letter, \$ or with a _
- Names are case sensitive - *myVar* and *Myvar* are different)
- Reserved words (like *class*, *int* or *boolean*) cannot be used as names
- Valid – *student_123*, *coursE_Name\$*, *_khalid*, *\$total*
- Invalid names – *1student*, *@name*, *hello!*, *my-var*, *class*

Datatypes in Java

- Data types in Java are divided into two groups:
- Primitive data types
 - byte, short, int, long, float, double, boolean and char
- Non-primitive data types
 - [String](#), and other Java [Classes](#) (you will learn more about them in CPCS 203)

Datatypes in Java – size and range

Data Type	Size	Description
<i>byte</i>	1 byte	Stores whole numbers from -128 to 127
<i>short</i>	2 bytes	Stores whole numbers from -32,768 to 32,767
<i>int</i>	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
<i>long</i>	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
<i>float</i>	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
<i>double</i>	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
<i>boolean</i>	1 bit	Stores true or false values
<i>char</i>	2 bytes	Stores a single character/letter or ASCII values (eg. 'A', '\$', '6', '@', 'h')
<i>String</i>	-	Stores multiple characters/letters (eg. "Khalid", "CS@FCIT", "BYG-4567")

Source : https://www.w3schools.com/java/java_data_types.asp

Datatypes in Java - Assignment

Data Type	Assignment
<i>byte</i>	<i>byte hour = 5;</i>
<i>short</i>	<i>short minutes = 123;</i>
<i>int</i>	<i>int temperature = -34;</i>
<i>long</i>	<i>long big = -9223854734475L; // the letter l or L has to be added for a long number</i>
<i>float</i>	<i>float temp_in_celcius = -34.545f; // the letter f or F has to be added for a float</i>
<i>double</i>	<i>double PI = 3.14159;</i>
<i>boolean</i>	<i>boolean answer=true;</i>
<i>char</i>	<i>char grade = 'B';</i>
<i>String</i>	<i>String name = "Khalid";</i>

Source : https://www.w3schools.com/java/java_data_types.asp

Declaring and initializing variables

```
// declare and initialize in one statement
```

```
double ax = 56.45, ay = -4.600005;
```

```
// first declare then initialize
```

```
float a;
```

```
a = 156.43f;
```

```
// initializing multiple variables. all will have the value 34
```

```
int a1, a2, a3, a4, a5;
```

```
a1 = a2 = a3 = a4 = a5 = 34;
```

```
// Syntax error. Assigning decimal value to int
```

```
int error_x = 56.45;
```

Constants in Java

- It is a variable whose value cannot be changed once it has been assigned
- They are declared using Java's **final** keyword
- The Java *naming convention* specifies that constant variables must be capitalized
- This makes programs easier to read and understand

Example,

```
final double PI = 3.14159;  
final int MAX_SPEED = 140;
```

Java Type Casting

Type casting is when you assign a value of one primitive data type to another type.

In Java, there are two types of casting:

1. Widening Casting - converting a smaller type to a larger type size

byte → short → char → int → long → float → double

2. Narrowing Casting - converting a larger type to a smaller size type

double → float → long → int → char → short → byte

Source : https://www.w3schools.com/java/java_type_casting.asp

Java Type Casting - Examples

```
byte b = 34;  
double d = 45.53;  
int i = b;           // Widening casting  
float f = i;  
int a = (int) 56.34; // Narrowing casting  
  
System.out.println(a);  
System.out.println((double) b); // Widening casting  
System.out.println((int)d + "\n"); // Narrowing casting  
System.out.println(22/7);  
System.out.println((float)22/7);  
System.out.println((double)22/7);
```

Output

```
56  
34.0  
45  
  
3  
3.142857  
3.142857142857143
```

Integer Division

- In Java, if any one of the operands in a division operation is a **double**, then "real" division is performed and the result is a **double**
- However, if both operands have type **int**, then the result is an **int** and the fraction part is ignored
- For example, the result of evaluating $(7 / 3)$ is 2
- Integer division is very useful when making change and in time conversion problems
- What if you want the answer as a **double**? Use typecasting.
- For example, `double(7)/3` or `7.0/3`

ASCII and Unicode (to represent characters)

character	ASCII value	character	ASCII value	character	ASCII value
<i>a</i>	97	A	65	0	48
<i>z</i>	122	Z	90	9	57
@	64	%	37	ESC (Escape)	27

Source : <https://www.rapidtables.com/code/text/ascii-table.html>

character	Unicode value	character	Unicode value	character	Unicode value
<i>a</i>	\u0061	A	\u0041	0	\u0030
<i>z</i>	\u007a	Z	\u005A	9	\u0039
Æ	\u00C6	∞ (infinity)	\u221E	¥ Japanese (Yen)	\u00A5
α (alpha)	\u03B1	ظ	\u0638	🚫	\u26D4

Source : <https://www.rapidtables.com/code/text/unicode-characters.html>

ASCII and Unicode – Output Examples

```
// Examples of using ASCII  
System.out.println((char)98);  
System.out.println((int)'b' + '\n');
```

```
// Examples of using Unicode  
System.out.println('\u0062C');  
System.out.println('\u00634');  
System.out.println('\u26BD');  
System.out.println('\u26FA');
```

Output

b
98

ج
ش



Operators – Arithmetic

Operator	Description	Example	ans
+	Adds together two values	ans = x + y	8
-	Subtracts one value from another	ans = x - y	2
*	Multiplies two values	ans = x * y	15
/	Divides one value by another	ans = x / y	1
%	Returns the division remainder	ans = x % y	2
+=	Addition assignment operators	ans += 1	3
-=	Subtraction assignment operators	ans -= 1	1
*=	Multiplication assignment operators	ans *= 2	6
/=	Division assignment operators	ans /= 2	1

Assume that **ans** is an *int* with value **2** and the initial values of **x** and **y** are **5** and **3**, respectively.

(source https://www.w3schools.com/java/java_operators.asp)

Reading from Console

Java uses the **Scanner** class to read from console. Two statements need to be added before we can begin reading from the console

1. We need to first import the Java Scanner Class using the following statement

```
import java.util.Scanner;
```

2. We need to initialize a Scanner object

```
Scanner in = new Scanner(System.in);
```

3. We need to choose the appropriate **Scanner** method based on the datatype to be read from the keyboard (next slide)

Reading from console – Methods to use

Data Type	method to read the datatype from console
<i>byte</i>	<i>nextByte()</i>
<i>short</i>	<i>nextShort()</i>
<i>int</i>	<i>nextInt()</i>
<i>long</i>	<i>nextLong()</i>
<i>float</i>	<i>nextFloat()</i>
<i>double</i>	<i>nextDouble()</i>
<i>boolean</i>	<i>nextBoolean()</i>
<i>String</i>	<i>next()</i> or <i>nextLine()</i>
<i>char</i>	<i>next.charAt(0)</i>

Scanner Methods to use - Examples

```
import java.util.Scanner;                // import the Java Scanner Class

public static void main(String[] args) {
    Scanner in = new Scanner(System.in); // initialize one Scanner object

    byte minutes = in.nextByte();        // 23
    short distance = in.nextShort();      // 125
    int number = in.nextInt();            // -4523
    long zipcode = in.nextLong();         // 4565323L
    float temp_in_Celcius = in.nextFloat(); // 45.63f
    double temp_in_Fahrenheit = in.nextDouble(); // 12454.5444545
    boolean answer = in.nextBoolean();    // true
    String name = in.next();               // Abdulrahman
    String fullName = in.nextLine();       // Anas Abdulrahman AlRajhi
    char grade = in.next().charAt(0);      // C
}
```

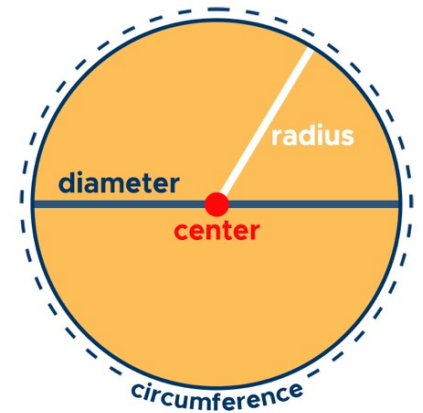
Example #1 – Calculate area of a circle

Problem statement

For a circle of a radius = 12.5, calculate and display its area and circumference. Assume the value of π (Pi) as 3.14159.

Area of a circle is given as : $Area = \pi r^2$

Circumference is given by : $Circumference = 2\pi r$



Example #1 – pseudo code

```
1: radius = 12.5
2: area = radius * radius * 3.14159
3: print "Area of circle is ", area
4: circumference = 2 * 3.14159 * radius
5: print "Circumference of circle is ",
    circumference
6:
```

Example #1 – Calculate area of a circle

```
double radius = 12.5;
final double PI = 3.14159;

double area = radius * radius * PI;
System.out.println("Area of circle is " + area);

double circumference = 2 * PI * radius;
System.out.println("Circumference of circle is " + circumference);
```

Output

```
Area of circle is 490.87343749999997
Circumference of circle is 78.53975
```

Example #2 – Converting Fahrenheit to Celsius

Problem statement

Write a program that reads a temperature in Fahrenheit from the user. It should then convert this temperature given in Fahrenheit into its Celsius equivalent. The following formula is used for the conversion

$$\text{Celsius} = \frac{5}{9} (\text{Fahrenheit} - 32)$$

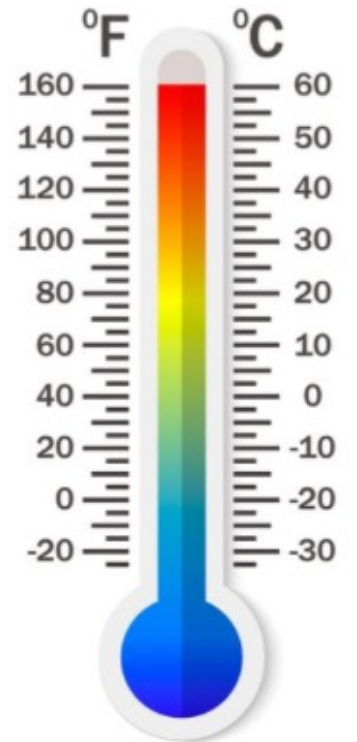
Hint : You have to use type casting to avoid an error in conversion

Example #2 – Solution in Java

```
System.out.print("Enter degree in Farenheit : ");  
double fahrenheit = in.nextDouble();  
  
double celcius = ((double)5/9) * (fahrenheit-32);  
// double celcius = (5.0/9) * (fahrenheit-32);  
  
System.out.println("Temp in Celcius is : " + celcius);
```

Output

```
Enter degree in Farenheit : 97.8  
Temp in Celcius is : 36.555555555555556
```



Example #3 – Making Change at Store

Problem statement

Assume you have notes in the following denominations to pay for an item at a grocery store – SAR 100, SAR 50, SAR 10, SAR 5 and SAR 1. Given an amount to pay at a store, list out the number of notes needed for each denomination to pay for the item at the counter.



You have done this numerous times at the grocery store. Work alone or in a group and design the algorithm (steps to solve this problem) using pseudo code before looking at the solution.

Example #3 – Solution in Java

```
System.out.print("Enter price of item : ");
int price = in.nextInt();

int hundreds = price / 100;
int remaining = price % 100;
int fifty = remaining / 50;
remaining %= remaining % 50;
int tens = remaining / 10;
remaining %= 10; // using short hand operator
int fives = remaining / 5;
int ones %= 5; // using short hand operator

System.out.println(hundreds + " x 100");
System.out.println(fifty + " x 50");
System.out.println(tens + " x 10");
System.out.println(fives + " x 5");
System.out.println(ones + " x 1");
```

Output

```
Enter price of item : 398
3 x SAR 100
1 x SAR 50
4 x SAR 10
1 x SAR 5
3 x SAR 1
```

Example #4 – Time conversion

Problem statement

Given time in seconds, convert it into its equivalent in hours, minutes and seconds.

For example, 10,000 seconds can be expressed 2:46:40 or 2 hours, 46 minutes and 40 seconds.

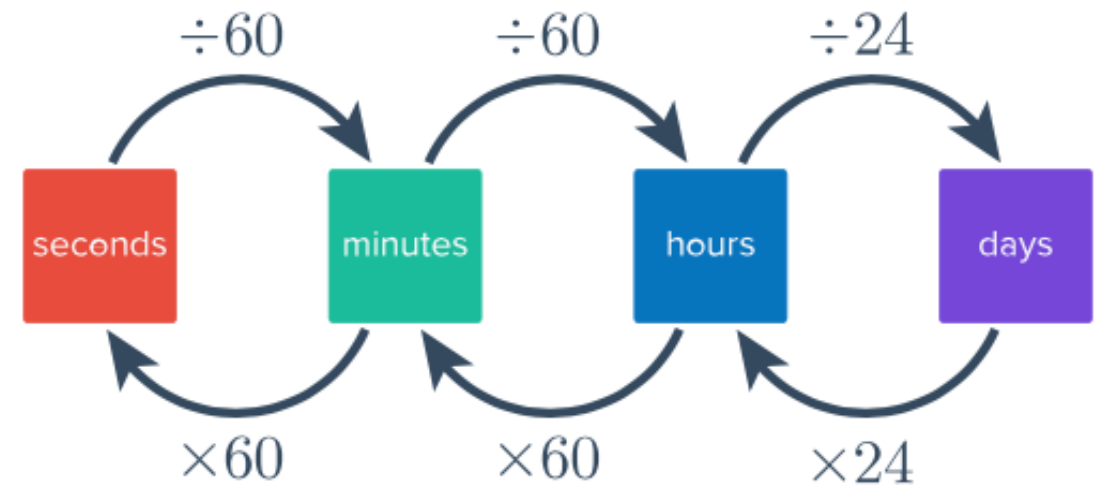


Image Source : <https://mathspace.co/textbooks/syllabuses/Syllabus-1081/topics/Topic-21041/subtopics/Subtopic-272983/>

Example #4 – Solution in Java

```
System.out.print("Enter time in seconds : ");  
int seconds = in.nextInt();  
  
int hours = seconds / 3600;  
int remaining = seconds % 3600;  
int minutes = remaining / 60;  
seconds = remaining % 60;  
  
System.out.print("Its equivalent is ");  
System.out.println(hours + ":" + minutes + ":" + seconds);
```

Output

Enter time in seconds : 10000
Its equivalent is 2:46:40

Example #5 – Price, VAT and Total

Problem statement

Given the price of an item, calculate and print its VAT and the total amount to 2 decimal places. (Not approximated)

Note : VAT is 15% of the price of an item



Image Source : <https://smartasset.com/taxes/value-added-tax>

Example #5 – Price, VAT and Total

```
Scanner in = new Scanner(System.in);

System.out.print("Enter price of item (in SAR) : ");
double price = in.nextDouble();

double vat = 0.15 * price;
double total = price + vat;

System.out.println("VAT : SAR " + vat);

total = (int)(total*100);
total = total / 100;
System.out.println("Total : SAR " + total);
```

Output

```
Enter price of item (in SAR) : 56.78
VAT : SAR 8.517
Total : SAR 65.29
```


Example #6 – Name, family name and initials

Problem statement

Read the grade, first name and family name of a student and print his grade and initials. Also, print the length of the family name.

For example, initials for Ammar Yasir are AY.

Hint : Read the name and family name as two separate strings



Image Source : <https://eightladieswriting.files.wordpress.com/2014/11/family-tree.jpg>

Example #6 – Name, family name and initials

```
System.out.print("Enter your semester grade : ");
char grade = in.next().charAt(0);
System.out.println("Your grade : " + grade);

System.out.print("Enter first name : ");
String firstName = in.next();
System.out.print("Enter family name : ");
String familyName = in.next();

String initials = firstName.charAt(0) + "" + familyName.charAt(0);
System.out.println("Your initials : " + initials);

System.out.println("Your family name has " +
    familyName.length() + " characters");
```

Output

```
Enter your semester grade : B
Enter first name : Ammar
Enter family name : Yasir
Your grade : B
Your initials : AY
Your family name has 5 characters
```

Increment and Decrement Operators

- In Java, the increment operator `++` increases the value of a variable by 1. Similarly, the decrement operator `--` decreases the value of a variable by 1
- If you use the `++` operator as a prefix like: `++var`, the value of `var` is incremented by 1; then it returns the value
- If you use the `++` operator as a postfix like: `var++`, the original value of `var` is returned first; then `var` is incremented by 1
- The `--` operator works in a similar way to the `++` operator except `--` decreases the value by 1

Increment and Decrement Operators - Examples

```
int var1 = 5, var2 = 5;

// First value of var1 (5) is displayed
// Then, it is increased to 6.
System.out.println("var1 : " + var1++);

// First var2 is increased to 6
// Then, its value (6) is displayed.
System.out.println("var2 : " + ++var2);

int x = var1++; // var1 is first assigned then incremented
int y = ++var2; // var2 is first incremented then assigned

System.out.println("x : " + x);
System.out.println("y : " + y);
```

Output

```
var1 : 5
var2 : 6
x : 6
y : 7
```

Increment and Decrement Operators - Examples

```
char letter1 = 'a';  
char letter2 = 'G';  
System.out.println(letter1++);  
System.out.println(--letter2);  
  
// Never write code like this!!!  
int x = 5, y = 6;  
int crazy = x++ + --y + ++x - --y;  
System.out.println(crazy);
```

Output

```
a  
F  
13
```

Operators – Relational and Logical

Operator	Description	Example	Result
<code>==</code>	Equal to	<code>x == y</code>	<i>false</i>
<code>!=</code>	Not equal	<code>x != y</code>	<i>true</i>
<code>></code>	Greater than	<code>x > y</code>	<i>true</i>
<code><</code>	Less than	<code>x < y</code>	<i>false</i>
<code>>=</code>	Greater than or equal to	<code>x >= y</code>	<i>true</i>
<code><=</code>	Less than or equal to	<code>x <= y</code>	<i>false</i>
<code>&&</code>	Logical and (true if both statements are true)	<code>(x < 5) && (x < 10)</code>	<i>false</i>
<code> </code>	Logical or (true if one of the statements is true)	<code>(x < 5) (x < 4)</code>	<i>true</i>
<code>!</code>	Logical not (Reverse the result)	<code>!(x < 5 && x < 10)</code>	<i>true</i>

Assume that the initial values of `x` and `y` are `5` and `3`, respectively.

(source https://www.w3schools.com/java/java_operators.asp)

Relational and Logical operators - Examples

```
System.out.print("Enter your grade : ");
char grade = in.next().charAt(0);

System.out.print("Enter your age : ");
int age = in.nextInt();

boolean pass = (grade=='A') || (grade=='B') ||
               (grade=='C') || (grade=='D');
boolean teen = (age>=13) && (age<=19);

System.out.println("Pass : " + pass);
System.out.println("Teenager : " + teen);
```

Output

```
Enter your grade : C
Enter your age : 35
Pass : true
Teenager : false
```